Omnichannel Pharmacy Operations Mobile Web Platform

CASE STUDY

1. Executive Summary

This project presents a scalable, offline-capable microservices-based web platform designed for regional pharmacy retail systems. The system is built to address real world retail challenges including high transaction volumes, intermittent connectivity, inventory consistency, operational visibility with integrated AI-driven decision support.

1.1. Key Highlights:

- **Domain-Driven Design** Microservices Architecture
- **Offline-first** Store operations
- **Scalable** and **Observable** System Design
- **AI-driven** Operational Intelligence
- **Security** and **Access control**

1.2. Prototype & Research Repository

A working prototype is currently in progress to demonstrate core microservices interaction and POS checkout flow.
GitHub Repository: https://github.com/Rudr-Praneeth/Centific-Case_Study
The repository also contains:

- Field research from pharma store visits: MedPlus, Apollo Pharmacy
- System design notes and architecture plans
- Supporting references.

2. PROBLEM DESCIPTION

2.1. Problem Statement

Design and develop a responsive **Microservices-based Web Application** for a regional pharmacy retailer operating across **185 stores, 2 central warehouses, and a growing e-commerce fulfilment network**. The solution should support secure authentication and role-based access

2.2. Functional Requirements

The functional requirements of the project include the following:

- Product and prescription-adjacent inventory tracking
- Point-of-sale sales and billing workflows
- Real-time stock visibility with replenishment alerts

- Store-level and warehouse-level reconciliation
- BI reporting dashboards for operational and commercial decision-making.
- Agentic AI features
- Behaviour Based access monitoring. (*New device login alters*)
- Designed for offline-tolerant workflows and reliable sync once connectivity is restored.

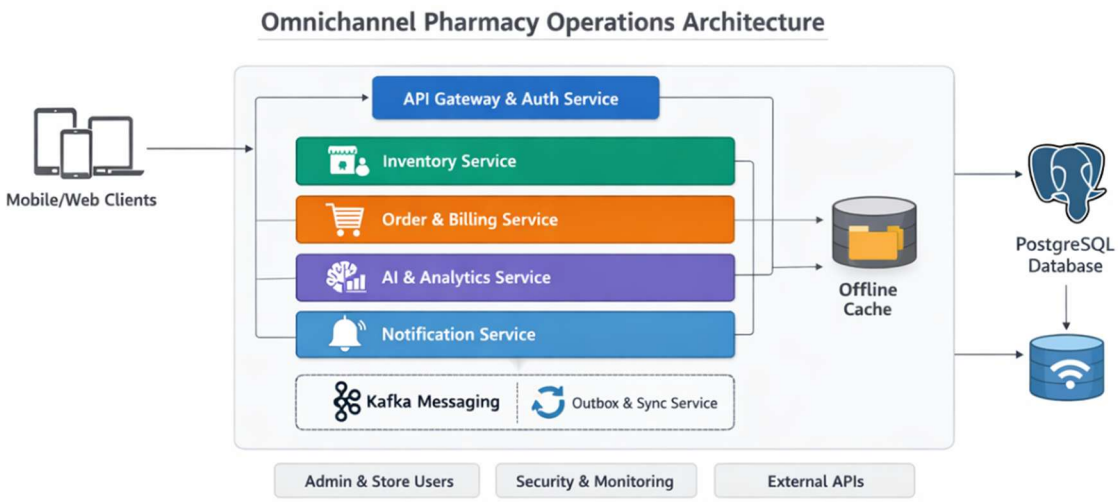
2.3. Non-Functional Requirements

The non-functional requirements of the project include the following:

- Performance under peak activity.
- Scalability across regions
- Security and Auditability
- Reliability
- Observability
- Responsible AI

3. ARCHITECTURE

3.1. High Level Architecture



3.2. API Gateway Layer

Kong API Gateway is used for centralised routing, authentication enforcement, rate limiting and request aggregation across services.

3.3. Microservices

The System design for the project follows a Domain-Driven Design (**DDD**) with separate data ownership i.e. decentralised data management and business logic per service. These services communicate with each other using APIs.

3.3.1. Service Consolidation Strategy

- Commerce Service: combining POS Billing and Catalog Pricing services
- Inventory Service: combining Inventory Replenishment and Inventory Reconciliation services.

- Identity Policy Service
- Platform Services: combining Sync service, BI service and Agentic AI Insights Service.

3.3.2. Identity Policy Service

Identity Policy Service is the micro service that owns user metadata, store mapping, device trust and access policies.
Authentication for the system is handled via the use of **Keycloak** which is a Identity and Access Management (IAM) server with Single Sign-On (SSO) support. Outsourcing identity to Keycloak helps delegate key tasks related to user authentication like password storage, hashing, password resets, Multi-Factor Authentication (MFA) and social logins.

Key Responsibilities:

- User-store mapping
- Role enrichment
- Device trust scoring
- Behavioural Anomaly Detection

Key Data Models:

- UserProfile
- Device
- LoginEvent
- PolicyRule
- RiskScore

Key APIs:

- GET /user-content
- POST /device/register
- GET /risk-score

Events Generated:

- LOGIN_ANOMAL_DETECTED
- NEW_DEVICE_LOGIN

3.3.3. Catalog Pricing Service

Catalog Pricing Service is the microservice that owns product master, pricing rules, promotions and substitution mappings.
The key responsibility of this service is POS pricing lookup, promo optimisation and Agentic AI substitution prompts.

Key Data Models:

- Product

- SKU
- PriceBook
- Promotion
- SubstitutionMap
- TaxRule

Key APIs:

- GET /products/{sku}
- GET /pricing/{sku}
- POST /promotions
- GET /substitutions/{sku}

Events Produced:

- PRICE_UPDATED
- PROMOTION_CREATED

Caching Strategies:
Redis & precomputed store-level price cache can be equipped to cache price and substitutes decreasing the response time for data by eliminating pricing computation in checkout path.

Pricing Resolution:

Billing Price = ((BasePrice + Store Overrides) - PromotionDiscount) + ApplicableTax

3.3.4. Inventory Replenishment Service

Inventory Replenishment Service is the key service of the whole system. It owns the stock knowledge including the store stock, warehouse stock along with the batch/expiry knowledge.
Available Stock = OnHandStock - ReservedStock - BlockedStock (damaged/expired)
The key responsibilities of this service include

- Real-time stock tracking
- Batch/expiry tracking
- Stock adjustments
- Stock reservation management using TTL-based reservations with expiry and auto-release to ensure strong consistency during checkout
- Inter-store stock transfers
- Replenishment Triggers

Key Data Models:

- InventoryItem
- BatchLot
- StockLedger

- ReorderRule
- Reservation
- TransferOrder

Key APIs:

- GET /inventory/{sku}
- POST /adjustments
- POST /transfers
- GET /reorder-suggestions
- POST /replenishment-orders
- POST /reserve
- POST /release

Events Produced:

- INVENTORY_UPDATED
- LOW_STOCK_ALERT
- EXPIRY_ALERT
- TRANSFER_COMPLETED

Consumers:

- Agentic AI Service for forecasting
- BI Service for dashboard
- Reconciliation services

3.3.5. POS Billing Service

POS Billing Service is a critical microservice of the system its responsibilities include

- Handling and validating Cart Lifecycle
- Fetching pricing from **Catalog Pricing Service** and Billing & Invoice generation
- Validating and reserving stock using **Inventory Replenishment Service**
- Applying Discounts
- Payment handling
- Handling returns & refunds

Key Data Models:

- Cart
- SaleTransaction
- SaleLineItem
- Payment
- Invoice
- ReturnTransaction

Key APIs:

- POST /cart
- POST /checkout
- POST /payments
- POST /returns

Events Produced:

- POS_SALE_COMPLETED
- POS_RETURN_COMPLETED
- POS_VOIDED

Special Design Considerations:

- Offline first compatibility
- Idempotent checkout API with orchestrated flow:
  Reserve -> Pay -> commit/rollback

3.3.6. Inventory Reconciliation Services

Reconciliation Service is the microservice that is responsible for handling

- Snapshot comparison
- Variance Detection
- Shrinkage tracking

Key Data Models:

- StockSnapshots
- VarianceCase
- ReconciliationReport

Workflow:

- Capture System snapshot
- Compare with physical count
- Identify variance
- Create a case
- Assign for resolution

Events Generated:

- VARIANCE_DETECTED
- RECONCILIATION_COMPLETED

3.3.7. Reporting BI Service

Reporting BI Service is the microservice that helps in following:

- Creating aggregated reporting across stores, warehouses and regions
- Shows KPI dashboards for insights

- Precomputed Data marts for low latency query handling by preparing data in advance.

Key Responsibilities:

- Shows sales reports
- Tracks inventory movement
- Compares store performance
- Gives business insights

Data Flow:

- Data flows through Apache Kafka
- Stored in PostgreSQL

This microservice uses the Materialised views i.e. the precomputed tables of PostgreSQL for fast read access with periodic refresh and incremental updates.
This uses materialised views for reading optimisations with Command Query Responsibility Separation (CQRS) design pattern considered for future scaling needs. It uses Kafka on the write side and fast tables on the read side.

Key APIs:

- GET /dashboard/sales
- GET /dashboard/inventory
- GET /reports/export

Observability & Monitoring:
To ensure reliable metrics and data shown on the KPI dashboards we use the following tools:

- OpenTelemetry: enables distributed tracing across microservices
- End-to-end trace correlation using trace_id across services
- Prometheus: collects metrics
- Grafana: provides high level dashboards
- Grafana Loki: logs query parameters, execution time and failure reasons useful for practical debugging.
- SLOs defined for POS latency <1s, inventory read <200ms and sync processing throughput.

3.3.8. Agentic AI Insights Service

Agentic AI Insights service is introduced as a initially limited to key responsibilities with future scope for expansion. It is responsible for the AI enabled optimisations for the System. It includes the following:

- Demand forecasting
- Anomaly Detection for Inventory movement
- Conversational Insights for KPI dashboards

- Workforce and staffing optimisations

Data Inputs:

- Kafka streams
- BI read models
- Inventory history

Components:

- Feature Extractor
- Model Engine
- Inference API
- Pgvector: Embedding Store

Key APIs:

- GET /forecast/{sku}
- GET /urgency_score/{sku}
- GET /anomalies
- POST /queries

3.3.9. Sync Service

Sync Service is the most critical microservice for stores to handle network disconnection acting as a bridge between offline PWA and online server.
This enables efficient operation of our system in hybrid environments where stores have intermittent connectivity, requiring offline-tolerant workflows and reliable sync once connectivity is restored.

Key Responsibilities:

- Offline transaction ingestion
- Replay orchestration
- Conflict resolution using event versioning (per sku_id & store_id) with deterministic ordering (timestamp & sequence_id).
- Conflict classifications
    - Inventory conflicts are resolved via reconciliation workflows
    - Financial conflicts are routed for manual review queue.
- Idempotency enforcement to ensure system saves the sale and replay in the same transaction.
- Data is partitioned by store_id and sku to reduce write contention and enable horizontal scalability.
- Backpressure handling and rate limiting during bulk replay after connectivity restoration.

Key Data Models:

- SyncBatch

- SyncEvent
- ConflictLog
- ReplayStatus

Key APIs:

- POST /sync/batch
- GET /sync/status
- POST /sync/retry

Workflow:

- Receive batch
- Validate deterministic ordering using store_id, timestamp and sequence_id
- Deduplicate for Idempotency
- Apply conflict resolution strategy using event versioning and conflict classification
- Route events that fail to Dead Letter Queue
- Route to services
- Track result of replay

Events Generated:

- SYNC_COMPLETED
- SYNC_FAILED
- CONFLICT_CAUSED

3.4. INTER SERVICE COMMUNICATION PATTERNS

Communication Type	Usage	Use Case
REST	External APIs	Communication between UI and Server
REST	Service-to-Service	Non-critical synchronous communication between microservices
gRPC	Internal low-latency	High performance synchronous communication EXAMPLE: Communication between POS and Inventory
Kafka	Async Events	Event-driven communication between services
IndexedDB + Sync Service	Offline-first	Local storage when network down/disconnected and sync using Sync Service

3.5. CROSS-CUTTING CONCERNS

- **Idempotency** enforced using database backend deduplication and Outbox pattern for atomic writes and event publishing.

- **Distributed Locking**: Redis based locks are used in inventory adjustments and in inventory transfers between stores/warehouses.
- **Audit Logging**: All services emit audit events with immutable storage.
- **Consistency Model**: The system approaches a hybrid consistency approach by using strong consistency for checkout critical paths and eventual consistency for downstream systems.
- **Event Schema Governance**: All Kafka events follow versioned schemas which is managed using schema registry to ensure backward compatibility.

3.6. KEY ARCHITECTURE TRADE-OFFS

Decision	Benefit	Trade-Off
Strict microservice boundaries and coupling	Scalability with reliability	More APIs (Service-Service communication APIs)
Event-driven design	Decoupling	Debugging complexity
Offline-first POS	Resilience	Sync complexity and conflicts
CQRS for KPI Dashboards	Performance	Data duplication and redundancy

4. UI EXPERIENCE LAYER

The User Experience layer is the interface layer through which users interact with the system. The experience layer is build using:

- ReactJs + TypeScript
- Tailwindcss
- TanStack Query for server state management and caching
- Progressive Web Application
- Service working for offline caching and request interception
- IndexedDB for local persistence for offline transactions

5. USER FLOWS

5.1. POS Checkout Flow – Online Mode

5.1.1. Cart Creation and Validation

User Action: Cashier scans barcode/ searched for product.
UI Behaviour: Instant product display and addition to cart
Backend Interactions:

- GET /products/{sku}
- GET /pricing/{sku}
- POST /cart

5.1.2. Price & Promotion Resolution

System Behaviour: System applies store-specific pricings, promotions, tax rules.
Backend Interactions: POST /promotions

5.1.3. Checkout Initiation

User Action: Proceeds to checkout
Backend Interactions:

- GET /substitutions/{sku} (Substitute suggestions before checkout)
- POST /checkout

5.1.4. Payment Processing

User Action: Selects payments method and processes payment.
Backend Interactions: POST /payments
On Success: confirm reservation
On Failure: release reservation

5.1.5. Transaction Commit

System Actions:

- Persist transaction (POS DB) using transactional outbox pattern after payment authorisation and stock reservation. Final commit only after successful payment and reservation confirmation.
- Generate Bill/Invoice by POS Billing Service
- Emit event via Outbox pattern

5.1.6. Async Updates

Kafka Triggers:

- Inventory Service for stock deduction
- BI Service for dashboard updates
- AI Service demand signal

UI Response:

- Show success screen
- Reset cart

5.2. POS Checkout Flow – Offline Mode

5.2.1. Cart Creation

User Action: Cashier scans barcode/ searched for product.
UI Behaviour: Instant product display and addition to cart
Instead of backend interactions for stock and price we use versioned cached catalog and pricing in IndexedDB with TTL and last-sync timestamp validation.

5.2.2. Price & Promotion Resolution

System Behaviour: System applies store-specific pricings, promotions, tax rules. Instead of backend integrations IndexedDB cached data is used.

5.2.3. Checkout Initiation and Processing

User Action: Proceeds to checkout
Post data for item is stored in the IndexedDB

5.2.4. **UI Feedback**

UI updates locally and clears cart

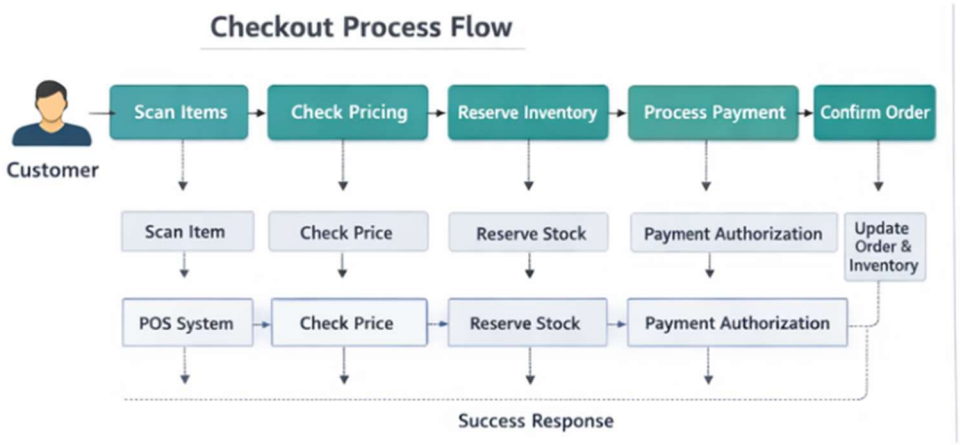
5.2.5. **Sync on Reconnect**

Trigger: Network restored
Flow: PWA connects to the Sync service
Backend Interactions: Sync Processing operations

- POST /sync/batch
- GET /sync/status
- POST /sync/retry

Events generated:

- SYNC_COMPLETED
- SYNC_FAILED
- CONFLICT_CAUSED



5.3. INVENTORY UPDATE FLOW

User Action
Inventory Controller can update data across stores, warehouses manually. Key operations include:

- Adjust stock
- Performs good receipts
- Initiates stock transfer between stores/warehouses.

Backend Interactions: POST /inventory/adjustments
Inventory Replenishment Service Processing:

- Writes into stock ledger
- Updates derived stock

Event Generated: INVENTORY_UPDATED
Downstream Events:

- BI Service updates dashboards
- AI Insights service updates demand signals

5.4. REPLENISHMENT WORKFLOW

5.4.1. Inventory Replenishment Trigger

Trigger: Low stock threshold breached
Event Generation Source: Inventory Replenishment Service
Event Generated: LOW_STOCK_ALERT

5.4.2. Event Processing and Display:

AI Service:

- Predicts demand
- Suggests replenishment reorder
- Calculates urgency score

Manager UI:

- Displays recommended order
- Urgency score

Backend Interactions:

- GET /reorder-suggestions

User Action: Approves order
Backend Call: POST /replenishment-orders

Warehouse receives order and processes dispatch.

API DESIGN

5.5. COMMON STANDARDS

Request Headers

Authorization: Bearer <JWT>
X-Store-Id: STR_001
X-Device-Id: DEV_001
X-Idempotency-Key: <optional>

5.6. API Endpoints:

Function	HTTP Method	API Endpoint	Service
Create Cart	POST	/cart	POS Billing Service
Checkout Initiation	POST	/checkout	POS Billing Service
Payment Initiation	POST	/payments	POS Billing Service
Return Item	POST	/returns	POS Billing Service
Get Inventory details including available stocks, batch_id, expiry & quantity	GET	/inventory/{sku}	Inventory Replenishment Service
Stock Adjustments	POST	/adjustments	Inventory Replenishment Service
Transfer stock between stores, between warehouses and between stores and warehouses	POST	/transfers	Inventory Replenishment Service
Replenish orders	POST	/replenishment-orders	Inventory Replenishment Service
Get Products for selected item	GET	/products/{sku}	Catalog Pricing Service
Get Price for selected item	GET	/pricing/{sku}	Catalog Pricing Service
Get User Context	GET	/user-content	Identity Policy Service
Register Device	POST	/device/register	Identity Policy Service
Get risk score for anomaly	GET	/risk-score	Identity Policy Service
Create Snapshot	POST	/snapshot	Inventory Reconciliation Service
Submit Physical count	POST	/reconciliation	Inventory Reconciliation Service
Get Variance Report	GET	/variance-report	Inventory Reconciliation Service
Get sales dashboard data	GET	/dashboard/sales	Reporting BI Service
Get inventory dashboard data	GET	/dashboard/inventory	Reporting BI Service

Get demand forecast for sku product	GET	/forecast/{sku}	AI Insights Service
Get Urgency score for product	GET	/urgency_score/{sku}	AI Insights Service
Get anomalies detected in cases of shrinkage and stock movement	GET	/anomalies	AI Insights Service
Conversational Querying for KPI dashboard insights	POST	/queries	AI Insights Service
Sync Batch Upload	POST	/sync/batch	Sync Service
Get Sync Status	GET	/sync/status	Sync Service
Retry Failed Sync	POST	/sync/retry	Sync Service

5.7. API Design Decisions and Trade-Offs

Decision	Benefit	Trade-off
Idempotent APIs	Safe retries	Extra storage overhead
Separate Sync APIs	Offline reliability	Conflict raises on longer network disconnection periods
Fine-grained services	Scalability	More API calls
Consistent response format	Easier debugging	(Working on setting a standard response statement which can be customised on a function level) Boilerplate overhead

6. Data Models

This section defines the system of record aligned with microservice data ownership, high write throughput, analytic readiness.

6.1. Data Modelling Principles

6.1.1. Database per Service

This refers to the logical isolation with each microservice has its own schema. There are no cross-service joins at database level. Communication occurs via APIs and Kafka.

6.1.2. Global Identifiers

The following global identifiers are used for correlation, event linking and analytics joins:

- store_id: Uniquely identifies the store
- sku: Uniquely identifies the specific sellable variant of a product
- transation_id: Uniquely identifies the transactions
- user_id: Uniquely identifies the user

6.2. **Core Entities**

Microservice	Database	Fields
Product and Catalog domain	Product	product_id, name, brand, category, is_active
Product and Catalog domain	Sku	Sku_id, product_id, pack_size, unit, mrp
Product and Catalog domain	Pricebook	price_id, sku_id, store_id, base_price
Product and Catalog domain	Promotion	Promo_id, sku_id, discount_value, start_date, end_date
Product and Catalog domain	SubstitutionMap	Sku_id, sub_sku
Inventory Replenishment Domain	InventoryItem	sku_id, location_id, available_stock, updated_at
Inventory Replenishment Domain	BatchLot	batch_id, sku_id, location_id, expiry_date, quantity
Inventory Replenishment Domain	StockLedger	ledger_id, sku_id, location_id, change, reason
Inventory Replenishment Domain	ReorderRule	sku_id, store_id, min_level, max_level, reorder_point
Inventory Replenishment Domain	TransferOrder	transfer_id, from_location, to_location, status
POS Billing Service	Cart	Cart_id, store_id, status
POS Billing Service	SaleTransaction	Transaction_id, store_id, amt, status
POS Billing Service	Payment	Payment_id, transaction_id, payment_mode, amt, status
POS Billing Service	Invoice	Invoice_id, transaction_id, invoice_number
POS Billing Service	ReturnTransaction	Return_id, transaction_id
Identity Policy Service	UserProfile	User_id, store_id, role
Identity Policy Service	DeviceProfile	Device_id, user_id, trusted, last_login
Identity Policy Service	LoginEvent	Event_id, user_id, device_id, ip_address
Inventory Reconciliation Service	StockSnapshot	Snapshot_id, store_id, created_at
Inventory Reconciliation Service	VarianceCase	Case_id, store_id, system_qty, physical_qty, variance, status
Inventory Reconciliation Service	ReconciliationReport	Report_id, store_id, summary
Sync Service	SyncBatch	Batch_id, store_id, status, created_at

Sync Service	SyncEvent	Event_id, batch_id, idempotency_key, type, status
Sync Service	ConflictLog	Conflict_id, event_id, cause, status
Sync Service	ReplayLog	Event_id, status, retry_count

6.3. **Caching Strategy**

Data	TTL
SKU Pricing	5 minutes
Product metadata	1 Hour
User session content	Short-lived
Inventory	3 seconds

6.4. **Data Model Trade offs**

Decision	Benefit	Trade-off
Ledger based inventory	Auditability	Storage overhead
No cross service joins	Scalability	Query Complexity increases
Partitioning	Performance	Operational overhead

7. **Agentic AI Features**

7.1. **Agentic Architecture Overview**

Core Components:

- Context Builder: Builds and extracts features from the data sources.
- Agent Engine: Combines LLM reasoning and ML models to generate insights and make decisions
- Tool/API Layer: Provides controlled access to system APIs used by AI Agents
- Memory Layer: Maintains long and short-term memories for better reasoning
- Human-in-loop Inference: Ensures critical decisions are in manager power.

Scope Limitation: Initial implementation limited to demand forecasting, replenishment recommendations.

7.2. **Demand Forecasting Pipeline**

Objective: Predict SKU level demand and drive replenishment decisions
Data Sources:

- Kafka events
- BI read models

Actions:

- Recommend reorder quantity
- Rank SKU urgency
- Auto create draft replenishment orders.

Guardrails: All AI-driven actions require explicit human approval with audit logging and confidence scores.

7.3. **Inventory Anomaly Agent**
Objective: Detect and act on shrinkage, unusual stock movement and fraud patterns.
Data Sources: Kafka Events
Actions:
- Trigger reconciliation workflow
- Notify manager and escalate high-risk cases

7.4. **Conversational BI Agent**
Objective: Enables natural language insights with possible recommendations for optimisation.
Data Sources:
- Kafka events
- BI read models

Actions:

- Answer queries
- Explain trends
- Suggest actions

7.5. Workforce Optimisation Agent

Objective: Optimise staff hours and shift locations based on demand.
Data Inputs:

- POS load patterns
- Footfall trends
- Peak hours

Actions:

- Staffing recommendations
- Shift schedules

8. DEPLOYMENT APPROACH

8.1. KUBERNETES ARCHITECTURE

The system is deployed on a **multi-node Kubernetes cluster** with logical separate namespaces for Core Services (POS Billing Service, Inventory Replenishment Service, Catalog & Pricing Service), Support namespace (Sync Services, AI Insight services, BI Services) and Infra namespace for infrastructure components (Kafka, Redis, PostgreSQL).

8.2. **Service Scaling Strategy**

The system uses **Horizontal Pod Autoscaling (HPA)** to scale services based on CPU, memory and request load.

8.3. **CI/CD Pipeline**

The deployment pipeline follows a simple flow:

- Code Commit
- Build
- Linting (ESLint)
- Testing
- Docker image
- Deployment via Helm ensuring versioned and consistent deployments across environments.

The system can be deployed across multiple regions with **region-specific clusters** while storing and processing data locally within region to reduce latency.
Cross-region replication uses asynchronous data replication with region-local writes and eventual consistency across regions.

9. **NON-FUNCTIONAL CONTROLS**

9.1. **PERFORMANCE**

The system adopts various performance optimisation strategies like

- **Redis caching** of pricing, products and hot inventory
- **Async processing** for inventory updates, BI refresh and AI insights
- **Connection pooling** using PgBouncer for PostgreSQL
- **Path optimisations** for POS Billing service by using minimal synchronous calls to achieve <1s latency target

9.2. **SCALABILITY**

The system adopts the following strategies for scaling:

- **Horizontal Scaling** using Kubernetes HPA
- **Stateless microservices**
- **Kafka partitioning** on store_id to ensure ordering guarantees and horizontal scalability

9.3. **SECURITY**

The system adopts the following strategies for security:

- JWT-based **OIDC** via Keycloak
- Role Based Access Control and Attribute Based Access Control Policies adopted
- Device trust and risk scoring
- Rate limiting via Kong
- Data protection via encryption at REST
- All inter-service communication is secured via mTLS within cluster.

9.4. **RELIABILITY**

The system ensures reliability is achieved in possible failure scenarios. Circuit breakers implemented for all downstream service calls using retry with exponential backoff.

Scenario	**Solution Adopted**
Payment success but system failure	Resolved via idempotent retry using transaction_id
Reservation expiry before payment	Triggers compensation and retry flow
Duplication of checkout requests	Handled via idempotent keys
Partial sync replay	Tracked via replay status and retired safely
Kafka consumer lag	Monitored and auto-scaled consumers

9.5. **RESPONSIBLE AI**

Guardrails like no PII Exposure and query validation are adopted along with explainable outputs with confidence scores to achieve AI-safe system.

9.6. **OFFLINE RESILIENCE**

Offline resilience is achieved by using PWA with Service worker for request interception, IndexedDB for local storage of UI data and sales data use the Sync Service for transaction replay.
Conflicts occurred are handled using idempotency keys and manual fallback.